

Phase 2 - Users, Groups & Access Control

This document covers the provisioning of a role-based user and group structure, password aging policy, offboarding controls, and shared-directory permission demonstrations (SGID and sticky bit) built during Phase 2.

1. Groups Created

Three role-based groups were created to mirror a small team structure: sysadmins (administrative access via wheel), devteam (developer collaboration), and auditors (read-only oversight role).

```
auditors:x:1008:hassan
alaa:x:1009:
mostafa:x:1010:
hassan:x:1011:
temp_contractor:x:1012:
```

2. Users Provisioned

Five accounts were created, each mapped to a role. The id command was used to verify UID assignment, primary group, and secondary group membership for each:

```
alaa
uid=1005(alaa) gid=1009(alaa) groups=1009(alaa),10(wheel),1006(sysadmins)

mostafa
uid=1006(mostafa) gid=1010(mostafa) groups=1010(mostafa),1007(devteam)

sara
uid=1002(sara) gid=1002(sara) groups=1002(sara),1007(devteam)

hassan
uid=1007(hassan) gid=1011(hassan) groups=1011(hassan),1008(auditors)

temp_contractor
uid=1008(temp_contractor) gid=1012(temp_contractor) groups=1012(temp_contractor)
```

Each user carries the correct secondary group for their role: alaa in sysadmins + wheel (administrative), mostafa and sara in devteam, hassan in auditors, and temp_contractor with no elevated group membership at all.

```
lobnaahmed@server:~/rhel-secure-server-project/configs$ for u in alaa mostafa sara hassan temp_contractor; do
> do echo "$u"
> id "$u"
> done
alaa
uid=1005(alaa) gid=1009(alaa) groups=1009(alaa),10(wheel),1006(sysadmins)
mostafa
uid=1006(mostafa) gid=1010(mostafa) groups=1010(mostafa),1007(devteam)
sara
uid=1002(sara) gid=1002(sara) groups=1002(sara),1007(devteam)
hassan
uid=1007(hassan) gid=1011(hassan) groups=1011(hassan),1008(auditors)
temp_contractor
uid=1008(temp_contractor) gid=1012(temp_contractor) groups=1012(temp_contractor)
```

3. Password Aging Policy

A password aging policy was applied to active accounts using `chage`: a 1-day minimum between password changes, a 90-day maximum before a forced change, and a 7-day warning period before expiry. The `alaa` account was additionally force-expired (`chage -d 0`) to demonstrate the forced-reset mechanism.

```
Last password change           : password must be changed
Password expires               : password must be changed
Password inactive              : password must be changed
Account expires                : never
Minimum number of days between password change : 1
Maximum number of days between password change : 90
Number of days of warning before password expires : 7
```

4. Offboarding Control: `temp_contractor` Lockdown

The `temp_contractor` account represents a decommissioned or temporary user. Two independent controls were deliberately stacked rather than relying on a single mechanism:

```
usermod -L temp_contractor      # locks password authentication
usermod -s /sbin/nologin temp_contractor # blocks interactive shell access
```

Verification via `passwd -S` confirms the lock is active (the "L" flag):

```
temp_contractor L 2026-07-03 0 99999 7 -1
```

Note: These two controls are complementary, not redundant. A locked password with no shell restriction can still be reactivated and used the moment it's unlocked; a nologin shell alone does not stop password authentication succeeding at a lower level. Applying both is a deliberate defense-in-depth decision.

5. Shared Directory Access Control

5.1 SGID — `/srv/devteam`

SGID was applied to `/srv/devteam` so that any file created inside by a `devteam` member automatically inherits the `devteam` group, without requiring a manual `chgrp`:

```
drwxrws---. 2 root devteam 30 Jul  4 17:20 /srv/devteam
```

The "s" in the group-execute position confirms the SGID bit is active.

```
lobnaahmed@servera:~$ sudo mkdir -p /srv/devteam
lobnaahmed@servera:~$ sudo chown root:devteam /srv/devteam/
lobnaahmed@servera:~$ sudo chmod 2770 /srv/devteam/
lobnaahmed@servera:~$ ls -ld /srv/devteam/
drwxrws---. 2 root devteam 6 Jul  4 17:17 /srv/devteam/
```

5.2 Sticky Bit — `/srv/shared-drop`

A world-writable shared drop folder was created with the sticky bit set, allowing any user to create files but preventing users from deleting files they don't own:

```
drwxrwxrwt. 2 root root 51 Jul  4 17:26 /srv/shared-drop
```

The trailing "t" confirms the sticky bit is active. This was verified concretely: sara was blocked from deleting a file created by mostafa, receiving "Operation not permitted" despite the directory being world-writable.

```
lobnaahmed@servera:~$ sudo mkdir -p /srv/shared-drop
lobnaahmed@servera:~$ sudo chmod 1777 /srv/shared-drop/
lobnaahmed@servera:~$ ls -ld /srv/shared-drop/
drwxrwxrwt. 2 root root 6 Jul  4 17:24 /srv/shared-drop/
lobnaahmed@servera:~$ sudo -u mostafa touch /srv/shared-drop/mostafa_file.txt
lobnaahmed@servera:~$ sudo -u sara touch /srv/shared-drop/sara_file.txt
lobnaahmed@servera:~$ sudo -u sara rm /srv/shared-drop/mostafa_file.txt
rm: remove write-protected regular empty file '/srv/shared-drop/mostafa_file.txt'? y
rm: cannot remove '/srv/shared-drop/mostafa_file.txt': Operation not permitted
```